



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ

по лабораторной работе № 4

по курсу «Защита информации»

на тему: «Разработка алгоритма симметричного шифрования (DES)»

Студент ИУ7-71Б
(Группа)

(Подпись, дата)

Я. Р. Овчинников
(И. О. Фамилия)

Преподаватель

(Подпись, дата)

Ю. С. Руденкова
(И. О. Фамилия)

2025 г.

СОДЕРЖАНИЕ

ЗАДАНИЕ	3
ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	4
РЕАЛИЗАЦИЯ	8
ДЕМОНСТРАЦИЯ РАБОТЫ ПРОГРАММЫ	14

ЗАДАНИЕ

Цель работы

Разработка алгоритма симметричного шифрования (DES). Шифрование и расшифровка произвольного файла.

Задание

Реализовать программу шифрования симметричным алгоритмом DES. Обеспечить шифрование и расшифровку произвольного файла с использованием разработанной программы. Предусмотреть работу программы с пустым, однобайтовым файлом.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1. Виды симметричного шифрования

Симметричное шифрование — это метод криптографии, в котором для шифрования и расшифровки используется один и тот же секретный ключ. Существуют два основных вида:

1.1. Поточное (потокосное) шифрование

Поточное шифрование обрабатывает данные побитово или побайтово. Генератор создаёт поток ключевой последовательности, которая складывается с исходным текстом операцией XOR:

$$C_i = M_i \oplus K_i$$

Примеры: RC4, ChaCha20, Salsa20, A5/1.

Преимущества: высокая скорость, низкие требования к памяти, подходит для потоковых данных.

Недостатки: критична синхронизация, повторное использование ключа опасно.

1.2. Блочное шифрование

Блочное шифрование обрабатывает данные блоками фиксированного размера. Исходный текст разбивается на блоки, каждый блок проходит через серию раундов с подстановками и перестановками.

Примеры: DES (64-битные блоки), AES (128-битные блоки), 3DES, ГОСТ 28147-89.

Преимущества: высокая криптостойкость, возможность параллельной обработки, хорошо стандартизированы.

Недостатки: требуется дополнение данных до размера блока, необходимость выбора режима работы.

В данной работе реализован блочный шифр DES с размером блока 64 бита.

2. Алгоритм шифрования DES

DES (Data Encryption Standard) — симметричный алгоритм блочного шифрования, использующий 64-битные блоки данных и 56-битный ключ (64 бита с учетом битов четности).

2.1. Структура алгоритма

Алгоритм DES состоит из следующих этапов:

- 1) **Начальная перестановка (IP):** 64-битный блок переставляется согласно таблице IP.
- 2) **16 раундов Фейстеля:** Блок делится на левую (L) и правую (R) половины по 32 бита. Для каждого раунда i :

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

где F — функция Фейстеля, K_i — подключ раунда.

- 3) **Функция Фейстеля** включает:
 - Расширение E: $32 \rightarrow 48$ бит;
 - XOR с подключом: $E(R) \oplus K$;
 - S-блоки: $48 \rightarrow 32$ бита (8 S-блоков по $6 \rightarrow 4$ бита);
 - Перестановка P: перемешивание 32 бит.
- 4) **Конечная перестановка (FP):** Применяется обратная перестановка $FP = IP^{-1}$.

2.2. Генерация подключей

Из 64-битного ключа генерируются 16 подключей по 48 бит:

- 1) PC1: из 64 бит выбираются 56 значащих, делятся на C и D по 28 бит;
- 2) Для каждого раунда C и D циклически сдвигаются влево на 1 или 2 позиции;
- 3) PC2: из 56 бит выбираются 48 для подключа K_i .

2.3. Расшифровка

Расшифровка выполняется тем же алгоритмом с подключами в обратном порядке: $K_{16}, K_{15}, \dots, K_1$. Благодаря структуре Фейстеля операции симметричны.

3. Алгоритмы перестановки и подстановки

3.1. Перестановка (Permutation)

Перестановка переставляет биты входной последовательности в определённом порядке согласно таблице. Для входа $X = x_1x_2\dots x_n$ и таблицы $P = [p_1, p_2, \dots, p_m]$ выход:

$$y_i = x_{p_i}, \quad i = 1, 2, \dots, m$$

Примеры в DES:

- IP — начальная перестановка ($64 \rightarrow 64$ бит);
- E — расширение ($32 \rightarrow 48$ бит);
- P — перестановка после S-блоков ($32 \rightarrow 32$ бита);
- FP — конечная перестановка ($64 \rightarrow 64$ бит).

Свойства: линейность, обратимость, обеспечивают диффузию (распространение влияния каждого бита).

3.2. Подстановка (Substitution)

Подстановка — нелинейное преобразование, заменяющее входную последовательность на другую согласно таблице (S-box): $y = S(x)$.

S-блоки в DES: размерность $6 \rightarrow 4$ бита. Для входа $b_1b_2b_3b_4b_5b_6$:

- Строка: $row = b_1b_6$ (биты 1 и 6);
- Столбец: $col = b_2b_3b_4b_5$ (биты 2–5);
- Выход: 4-битное значение из таблицы $S[row][col]$.

Свойства: обеспечивают нелинейность и конфузию (скрывают связь между ключом и шифротекстом), устойчивы к линейному и дифференциальному криптоанализу.

3.3. Алгоритм, использующий оба подхода

DES комбинирует перестановки и подстановки (SP-сеть):

- **Перестановки** (IP, E, P, FP) обеспечивают **диффузию** — распространение влияния каждого бита на весь блок;
- **Подстановки** (S-блоки) обеспечивают **конфузию** — усложнение связи между ключом и шифротекстом;
- Чередование этих операций в 16 раундах создаёт высокую криптографическую стойкость.

РЕАЛИЗАЦИЯ

На листинге 1 приведена реализация функции генерации подключей для 16 раундов шифрования.

Листинг 1 – Генерация подключей

```
1 void DES::generate_subkeys() {
2     // Применяем PC1 для получения 56-битного ключа
3     uint64_t permuted_key = permute(key, DESTable::PC1, 64, 56);
4
5     // Разделяем на две половины по 28 бит
6     uint32_t C = (permuted_key >> 28) & 0xFFFFFFFF;
7     uint32_t D = permuted_key & 0xFFFFFFFF;
8
9     // Генерируем 16 подключей
10    for (int i = 0; i < 16; ++i) {
11        // Циклический сдвиг влево
12        C = left_shift(C, DESTable::SHIFTS[i], 28);
13        D = left_shift(D, DESTable::SHIFTS[i], 28);
14
15        // Объединяем C и D
16        uint64_t CD = ((uint64_t)C << 28) | D;
17
18        // Применяем PC2 для получения 48-битного подключа
19        subkeys[i] = permute(CD, DESTable::PC2, 56, 48);
20    }
21 }
```

На листинге 2 приведена реализация функции шифрования одного 64-битного блока с применением 16 раундов Фейстеля.

Листинг 2 – Шифрование блока

```
1 uint64_t DES::encrypt_block(uint64_t block) {
2     // Initial Permutation
3     block = permute(block, DESTable::IP, 64, 64);
4
5     // Разделяем на левую и правую части
6     uint32_t L = (block >> 32) & 0xFFFFFFFF;
7     uint32_t R = block & 0xFFFFFFFF;
8 }
```



```

9      // 16 раундов Фейстеля
10     for (int i = 0; i < 16; ++i) {
11         uint32_t temp = R;
12         R = L ^ feistel(R, subkeys[i]);
13         L = temp;
14     }
15
16     // Объединяем (R затем L для финальной перестановки)
17     uint64_t combined = ((uint64_t)R << 32) | L;
18
19     // Final Permutation
20     return permute(combined, DESTable::FP, 64, 64);
21 }

```

На листинге 3 приведена реализация функции Фейстеля — ядра алгоритма DES, включающей расширение, S-блоки и перестановку.

Листинг 3 – Функция Фейстеля

```

1  uint32_t DES::feistel(uint32_t R, uint64_t subkey) {
2      // Expansion (32 -> 48 бит)
3      uint64_t expanded = permute(R, DESTable::E, 32, 48);
4
5      // XOR с подключом
6      uint64_t xored = expanded ^ subkey;
7
8      // S-блоки (48 -> 32 бита)
9      uint32_t output = 0;
10     for (int i = 0; i < 8; ++i) {
11         uint8_t input = (xored >> (42 - 6*i)) & 0x3F;    //
12                     // Извлекаем 6 бит
13         uint8_t sbbox_out = sbbox_lookup(input, i);
14         output = (output << 4) | sbbox_out;
15     }
16
17     // Permutation P
18     return permute(output, DESTable::P, 32, 32);
19 }
20
21 uint8_t DES::sbox_lookup(uint8_t input, int box_num) {
22     int row = ((input & 0x20) >> 4) | (input & 0x01);    // Биты 1
23     // и 6

```

```

22     int col = (input >> 1) & 0x0F;                                // Биты
        2-5
23     return DESTable::S[box_num][row][col];
24 }

```

На листинге 4 приведена реализация функции шифрования файла с блочной обработкой данных и сохранением исходного размера.

Листинг 4 – Шифрование файла

```

1 void DES::encrypt_file(const std::string& input_path,
2                       const std::string& output_path) {
3     if (!key_generated) {
4         throw DESExceptions::KeyNotGenerated();
5     }
6
7     std::ifstream input(input_path, std::ios::binary);
8     if (!input.is_open()) {
9         throw DESExceptions::FileReadError(input_path);
10    }
11
12    // Получаем размер файла
13    input.seekg(0, std::ios::end);
14    uint64_t file_size = input.tellg();
15    input.seekg(0, std::ios::beg);
16
17    std::ofstream output(output_path, std::ios::binary);
18    if (!output.is_open()) {
19        throw DESExceptions::FileWriteError(output_path);
20    }
21
22    // Записываем оригинальный размер файла в начало
23    output.write(reinterpret_cast<const char*>(&file_size),
24                sizeof(file_size));
25
26    std::cout << "Шифрование файла...\n";
27    std::cout << "Размер файла: " << file_size << " байт\n";
28
29    // Читаем и шифруем блоками по 8 байт
30    uint8_t buffer[8];
31    size_t blocks_processed = 0;

```

```

32     while (true) {
33         input.read(reinterpret_cast<char*>(buffer), 8);
34         size_t bytes_read = input.gcount();
35
36         if (bytes_read == 0) break;
37
38         // Zero padding для последнего блока
39         if (bytes_read < 8) {
40             std::memset(buffer + bytes_read, 0, 8 - bytes_read);
41         }
42
43         // Преобразуем в uint64_t и шифруем
44         uint64_t block = bytes_to_uint64(buffer);
45         uint64_t encrypted = encrypt_block(block);
46
47         // Записываем зашифрованный блок
48         uint8_t encrypted_bytes[8];
49         uint64_to_bytes(encrypted, encrypted_bytes);
50         output.write(reinterpret_cast<const
51             char*>(encrypted_bytes), 8);
52
53         blocks_processed++;
54     }
55
56     std::cout << "\nШифрование завершено успешно!\n";
57     input.close();
58     output.close();
59 }

```

На листинге 5 приведена реализация функции расшифровки файла с восстановлением исходного размера данных.

Листинг 5 – Расшифровка файла

```

1 void DES::decrypt_file(const std::string& input_path,
2                       const std::string& output_path) {
3     if (!key_generated) {
4         throw DESExceptions::KeyNotGenerated();
5     }
6     std::ifstream input(input_path, std::ios::binary);
7     if (!input.is_open()) {
8         throw DESExceptions::FileReadError(input_path);
9     }

```

```

9      }
10     uint64_t original_size;
11     input.read(reinterpret_cast<char*>(&original_size),
12               sizeof(original_size));
13     if (!input) {
14         throw DESExceptions::DecryptionError("Неверный формат
15         зашифрованного файла");
16     }
17     std::ofstream output(output_path, std::ios::binary);
18     if (!output.is_open()) {
19         throw DESExceptions::FileWriteError(output_path);
20     }
21
22     std::cout << "Расшифровка файла...\n";
23     std::cout << "Оригинальный размер: " << original_size << "
24     байт\n";
25
26     uint64_t bytes_written = 0;
27     uint8_t buffer[8];
28
29     // Читаем и расшифровываем блоками по 8 байт
30     while (input.read(reinterpret_cast<char*>(buffer), 8)) {
31         // Преобразуем в uint64_t и расшифровываем
32         uint64_t block = bytes_to_uint64(buffer);
33         uint64_t decrypted = decrypt_block(block);
34
35         // Преобразуем обратно в байты
36         uint8_t decrypted_bytes[8];
37         uint64_to_bytes(decrypted, decrypted_bytes);
38
39         // Определяем сколько байт записать (для последнего
40         блока)
41         size_t bytes_to_write = 8;
42         if (bytes_written + 8 > original_size) {
43             bytes_to_write = original_size - bytes_written;
44         }
45
46         // Записываем расшифрованные байты
47         output.write(reinterpret_cast<const
48             char*>(decrypted_bytes), bytes_to_write);
49         bytes_written += bytes_to_write;

```

```
45     }
46
47     std::cout << "\nРасшифровка завершена успешно!\n";
48     input.close();
49     output.close();
50 }
```

ДЕМОНСТРАЦИЯ РАБОТЫ ПРОГРАММЫ

На листинге ниже продемонстрирован основной сценарий работы программы:

Листинг 6 – Основной сценарий работы программы

```
1  === Загрузка ключа ===
2  Введите имя файла ключа (без пути, в директории out/): 1
3  Ошибка: Невозможно прочитать файл ключа: out/1
4
5  === DES Шифрование ===
6  1. Сгенерировать ключ
7  2. Загрузить ключ из файла
8  3. Сохранить ключ в файл
9  4. Зашифровать файл
10 5. Расшифровать файл
11 6. Показать ключ
12 0. Выход
13 > 1
14
15 === Генерация ключа ===
16 Ключ успешно сгенерирован!
17
18 === DES Шифрование ===
19 1. Сгенерировать ключ
20 2. Загрузить ключ из файла
21 3. Сохранить ключ в файл
22 4. Зашифровать файл
23 5. Расшифровать файл
24 6. Показать ключ
25 0. Выход
26 > 3
27
28 === Сохранение ключа ===
29 Введите имя файла для ключа (без расширения): key
30 Ключ сохранен: out/key.key
31
32 === DES Шифрование ===
33 1. Сгенерировать ключ
34 2. Загрузить ключ из файла
35 3. Сохранить ключ в файл
36 4. Зашифровать файл
```

```

37 5. Расшифровать файл
38 6. Показать ключ
39 0. Выход
40 > 2
41
42 === Загрузка ключа ===
43 Введите имя файла ключа (без пути, в директории out/): key.key
44 Ключ успешно загружен
45
46 === DES Шифрование ===
47 1. Сгенерировать ключ
48 2. Загрузить ключ из файла
49 3. Сохранить ключ в файл
50 4. Зашифровать файл
51 5. Расшифровать файл
52 6. Показать ключ
53 0. Выход
54 > 4
55
56 === Шифрование файла ===
57
58 Доступные файлы:
59     1. empty.txt
60     2. single.txt
61
62 Выберите файл для шифрования (введите номер от 1 до 2, 0 для
    отмены): 2
63
64 Шифрование: single.txt -> single.txt.des
65 Шифрование файла...
66 Размер файла: 1 байт
67 Обработано блоков: 1/1
68 Шифрование завершено успешно!
69 Зашифрованный файл сохранен: out/single.txt.des
70
71 === DES Шифрование ===
72 1. Сгенерировать ключ
73 2. Загрузить ключ из файла
74 3. Сохранить ключ в файл
75 4. Зашифровать файл
76 5. Расшифровать файл

```

```

77 6. Показать ключ
78 0. Выход
79 > 5
80
81 === Расшифровка файла ===
82
83 Доступные файлы:
84 1. single.txt.des
85
86 Выберите файл для расшифровки (введите номер от 1 до 1, 0 для
    отмены): 1
87
88 Расшифровка: single.txt.des -> decrypted_single.txt
89 Расшифровка файла...
90 Оригинальный размер: 1 байт
91
92 Расшифровка завершена успешно!
93 Расшифрованный файл сохранен: out/decrypted_single.txt
94
95 === DES Шифрование ===
96 1. Сгенерировать ключ
97 2. Загрузить ключ из файла
98 3. Сохранить ключ в файл
99 4. Зашифровать файл
100 5. Расшифровать файл
101 6. Показать ключ
102 0. Выход
103 > 6
104
105 === DES Ключ ===
106 Ключ (hex): 48ce4c54978bb270

```

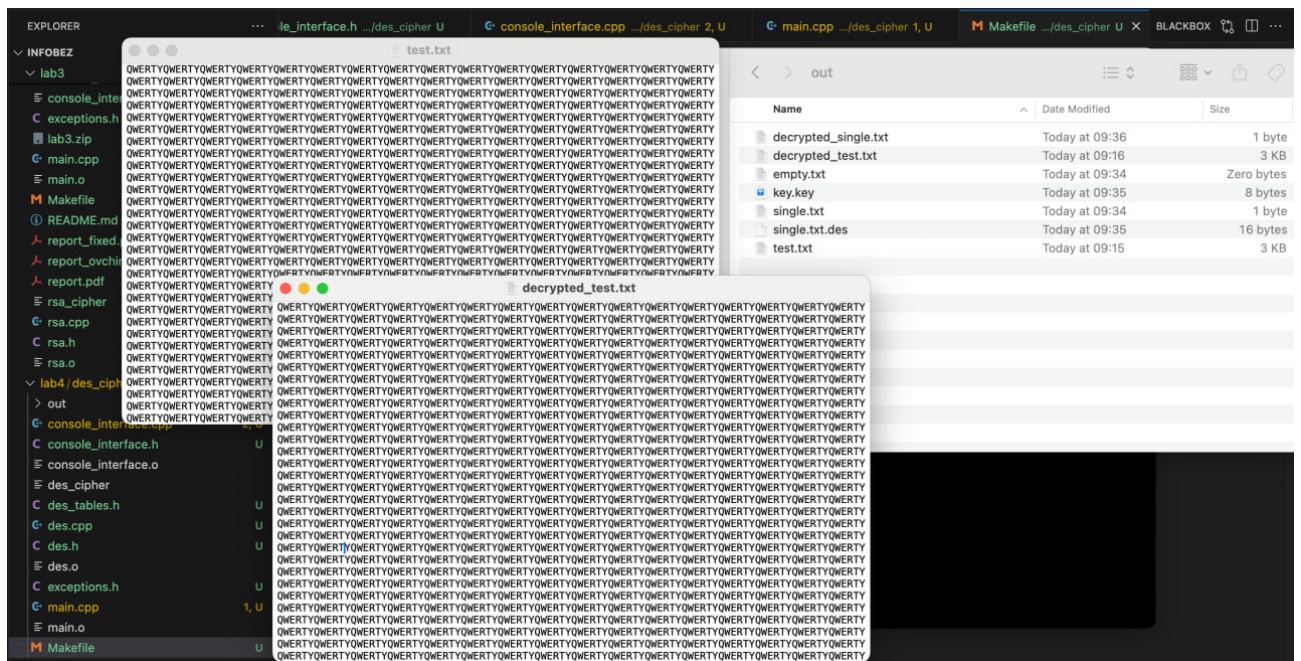



Рисунок 1 – Шифрование большого файла

Также была предусмотрена обработка большинства ошибочных ситуаций без экстренного завершения программы:

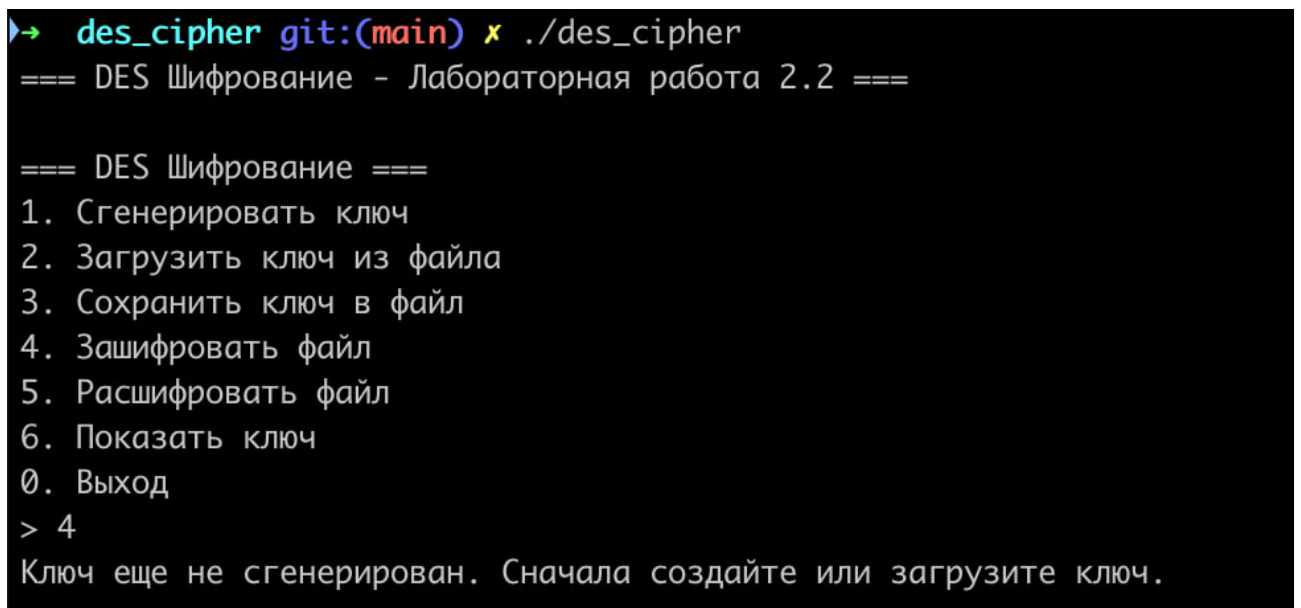


Рисунок 2 – Обработка ошибочных ситуаций